

# Web программирование

Курс для бакалавриата

Чернышев Вячеслав



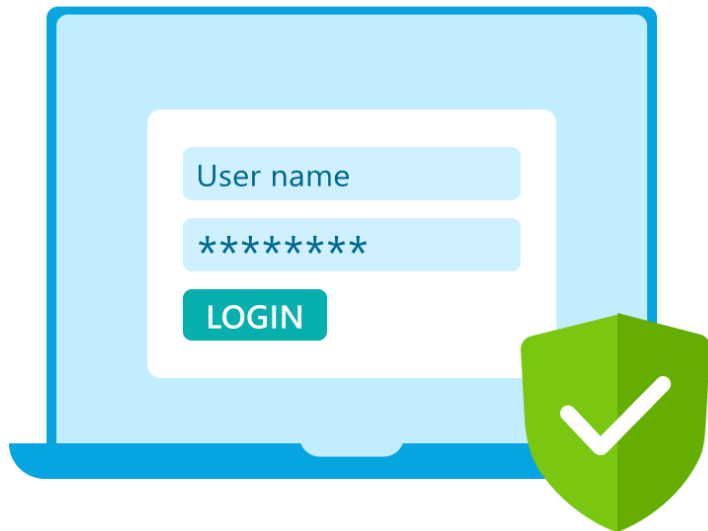
# Аутентификация и авторизация

Чернышев Вячеслав

# Содержание лекции

1. Аутентификация и авторизация
2. Виды авторизации
3. OAuth FastAPI
4. Hashing and salting
5. Refresh token
6. SSO
7. OWASP Top 10 API Security

## Аутентификация



Является ли пользователь тем,  
за кого себя выдаёт?

## Авторизация



Есть ли у пользователя разрешение  
на доступ к данным?

# Аутентификация

- Идентифицирует пользователя
- Предоставляет общий доступ в систему
- Работает в пределах сессии
- Идет перед авторизацией (по общему правилу, но не всегда)

# Авторизация

- Дает права к конкретному ресурсу (уровень доступа)
- Есть группы пользователей (роли)
- Идет после аутентификации (как правило)

# Виды авторизации в приложениях FastAPI



Cookie



OAuth



API  
Key



Basic Auth

# Протоколы авторизации

Отличаются уровнем сложности и безопасности

| Тип        | Безопасность | Сложность | Пример                                              |
|------------|--------------|-----------|-----------------------------------------------------|
| Cookie     |              |           | Работа с браузерами пользователей                   |
| OAuth      |              |           | Мобильные клиенты, внешние интеграции               |
| API Key    |              |           | Внутрисетевые межсервисные запросы                  |
| Basic Auth |              |           | Очень простые админ-эндпоинты за прокси, dev-стенды |

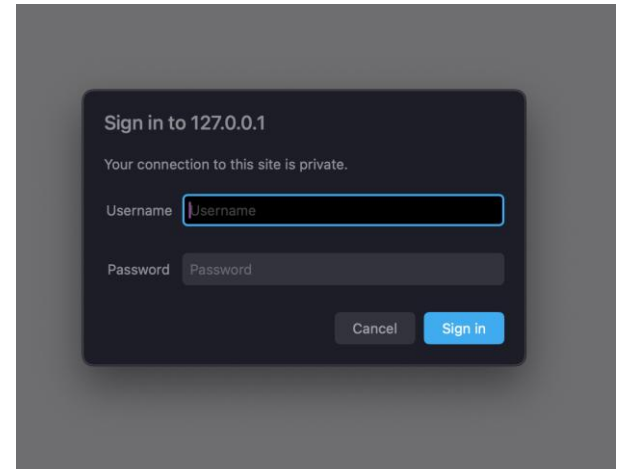


# Basic Auth

## Header

Authorization: Basic YWRtaW46c2VjcmV0 ('admin:secret' Encoded to base64)

```
curl -u myuser:mypassword https://mysite/private
```



# Cookie based

Позволяет работать только с браузерами

```
curl -c cookies.txt -X POST -H "Content-Type: application/json" \  
  -d '{"username": "admin", "password": "secret"}' \  
  http://127.0.0.1:8000/login
```

```
curl -b cookies.txt http://127.0.0.1:8000/private
```

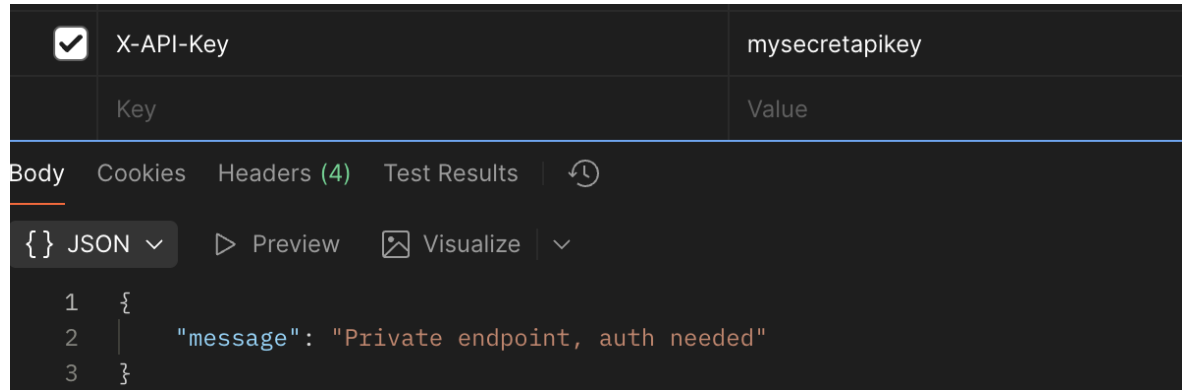
```
curl -b cookies.txt -X POST http://127.0.0.1:8000/logout
```

```
@app.post("/login")  
def login(data: LoginData, response: Response):  
    if data.username != 'admin' or data.password != 'secret':  
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid credentials")  
    token = "simple-session-token"  
    SESSIONS.add(token)  
  
    response.set_cookie(  
        key="session",  
        value=token,  
        httponly=True,  
        max_age=60 * 30,  
        samesite="lax"  
    )  
    return {"message": "Logged in successfully"}
```

# API Key

## Header

ANY-NAME: mysecretkey



# Environment variables

pip install python-dotenv

pip install pydantic-settings

.env файл должен быть в .gitignore!!!

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    API_KEY: str

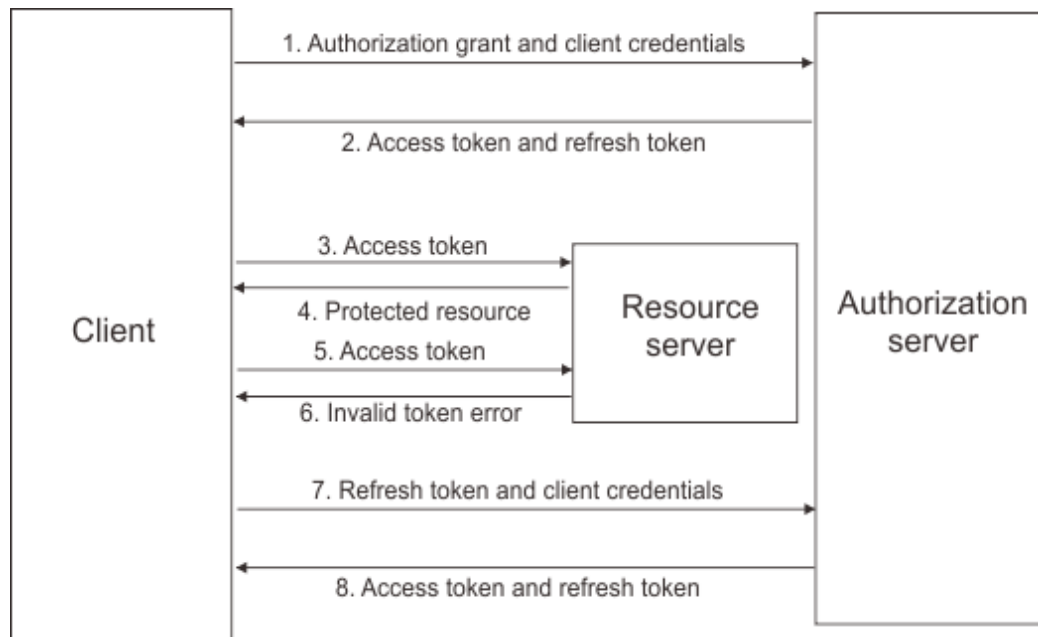
    class Config:
        env_file = ".env"

settings = Settings()

app = FastAPI()

API_KEY = settings.API_KEY
```

# Схема работы OAuth



<https://proglib.io/p/vyydi-i-snova-zaydi-tolko-pravilno-vse-li-vy-znaete-ob-oauth-2-0-2020-07-30>

# OAuth протокол

Использует:

- JWT
- Access/Refresh токены
- Шифрованная подпись

# JWT

## 1. Заголовки

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

## 1. JWS Payload

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "admin": true,  
  "iat": 1516239022  
}
```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNjM0NTY3ODkwXQ.KMUFsIDTnFmyG3nMiGM6H9FNFUOf3wh7SmqJp-QV3

## 1. Подпись

<https://www.jwt.io/>

# Особенности JWT

- Данные ключа - открыты
- Порядок ключей - важен
- Валидация ключей - только по подписи



# OAuth протокол

Использует:

- JWT
- Access/Refresh токены
- Шифрованная подпись

# Базовый пример OAuth FastAPI

 models.py

 oauth.py

 security.py

 storage.py

access-token >  security.py

```
1  DEFAULT_USERNAME = "user"
2  DEFAULT_PASSWORD = "password"
3
4  TOKEN_ALGORITHM = "HS256"
5  TOKEN_LIFETIME = 600
6  TOKEN_SECRET = "secret"
7
8  def validate_credentials(username: str, password: str) -> bool:
9      return username == DEFAULT_USERNAME and password == DEFAULT_PASSWORD
```

# Базовый пример OAuth FastAPI

access-token >  models.py

```
5     class User(BaseModel):
6         uid: UUID = Field(default_factory=uuid4)
7         username: str
8         is_active: bool = True
9         is_admin: bool = False
10
11
12     class AccessToken(BaseModel):
13         value: str
14         type: str = "Bearer"
15
16     class TokenPayload(User):
17         iat: datetime
18         exp: datetime
19
20         @field_serializer("iat", "exp", when_used="json")
21         def to_timestamp(self, value: datetime) -> datetime:
22             return datetime.timestamp(value.replace(microsecond=0))
```

# Базовый пример OAuth FastAPI

```
app = FastAPI()
oauth = OAuth2PasswordBearer(tokenUrl="token")
router = APIRouter()

def get_or_create_user(username: str) -> User:
    if username not in users_storage:
        users_storage[username] = User(username=username)
    return users_storage[username]

def create_access_token(username: str) -> str:
    payload = TokenPayload(
        username=username,
        iat=datetime.now(timezone.utc),
        exp=datetime.now(timezone.utc) + timedelta(minutes=TOKEN_LIFETIME)
    )
    return jwt.encode(jsonable_encoder(payload), TOKEN_SECRET, algorithm=TOKEN_ALGORITHM)
```

# Базовый пример OAuth FastAPI

pip install PyJWT

```
app = FastAPI()
oauth = OAuth2PasswordBearer(tokenUrl="token")
router = APIRouter()

def get_or_create_user(username: str) -> User:
    if username not in users_storage:
        users_storage[username] = User(username=username)
    return users_storage[username]

def create_access_token(username: str) -> str:
    payload = TokenPayload(
        username=username,
        iat=datetime.now(timezone.utc),
        exp=datetime.now(timezone.utc) + timedelta(minutes=TOKEN_LIFETIME)
    )
    return jwt.encode(jsonable_encoder(payload), TOKEN_SECRET, algorithm=TOKEN_ALGORITHM)
```

# Базовый пример OAuth FastAPI

```
@router.post("/token")
async def init_session(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> AccessToken:
    if not validate_credentials(form_data.username, form_data.password):
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN)

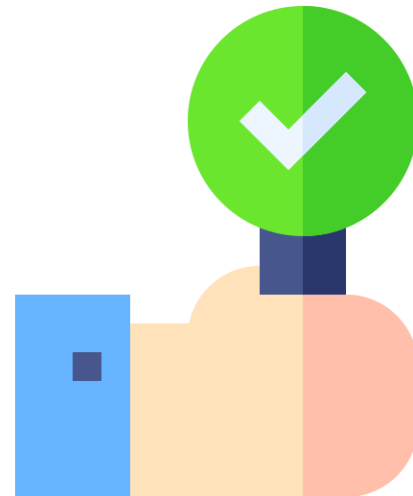
    token = create_access_token(username=form_data.username)
    return AccessToken(value=token)
```

# Базовый пример OAuth FastAPI

```
@router.get("/me")
async def get_current_user(request: Request, token: Annotated[str, Depends(oauth)]) -> User:
    try:
        payload = jwt.decode(token, TOKEN_SECRET, algorithms=[TOKEN_ALGORITHM])
        user = get_or_create_user(payload["username"])
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED)
    else:
        return user
```

# OAuth

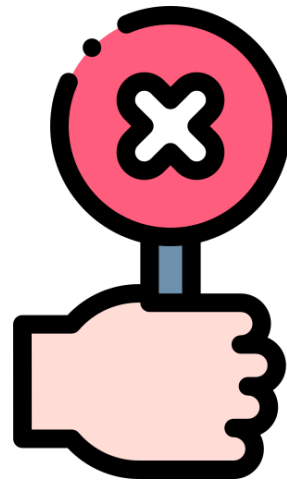
- Использовать для SPA-приложений и межсервисного взаимодействия
- Передавать в jwt открытые данные
- Проверять подпись
- Контролировать порядок ключей в данных (headers + payload)





# OAuth

- Использовать для браузерных приложений
- Передавать в jwt чувствительные данные
- Доверять любому JWT



# Получение данных пользователя в зависимости

 depends.py

 models.py

 oauth.py

 security.py

 storage.py

```
oauth = OAuth2PasswordBearer(tokenUrl="token")

async def get_oauth_user(
    token: Annotated[str, Depends(oauth)]
) -> User | None:
    try:
        payload = jwt.decode(token, TOKEN_SECRET, algorithms=[TOKEN_ALGORITHM])
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid token")
    user = users_storage.get(payload["username"])
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Not authorized")
    return user
```

## Получение данных пользователя в зависимости

```
@router.get("/me")
async def get_current_user(user: Annotated[User, Depends(get_oauth_user)]) -> User:
    return user
```

# RBAC модель

```
23  async def get_oauth_admin(  
24      user: Annotated[User, Depends(get_oauth_user)]  
25  ) -> User | None:  
26      if not user.is_admin:  
27          raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Admin access required")  
28      return user  
29
```

```
@router.get("/admin")  
async def get_admin_user(user: Annotated[User, Depends(get_oauth_admin)]) -> User:  
    return user
```

# Хранение паролей

- В БД нужно хранить хэшированные пароли
- Соль (salt) — случайная строка, добавляемая к паролю перед вычислением хеша. Нужна, чтобы одинаковые пароли имели разные хеши и чтобы затруднить радужные таблицы.
- Хэш необратим

# Хэширование пароля в python

- Argon2
- bcrypt

pip install bcrypt

```
import bcrypt

def hash_password(password: str) -> str:
    password_bytes = password.encode('utf-8')
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password_bytes, salt)
    return hashed.decode('utf-8')

def verify_password(password: str, hashed_password: str) -> bool:
    password_bytes = password.encode('utf-8')
    hashed_bytes = hashed_password.encode('utf-8')
    return bcrypt.checkpw(password_bytes, hashed_bytes)
```

# Хэширование пароля в python

pip install argon2-cffi

```
from argon2 import PasswordHasher

ph = PasswordHasher()

def hash_password(password: str) -> str:
    return ph.hash(password)

def verify_password(password: str, hashed_password: str) -> bool:
    try:
        ph.verify(hashed_password, password)
        return True
    except Exception:
        return False
```

# Сохранение пользователей

```
from database import Base
from sqlalchemy import Column, Integer, String, Boolean

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, autoincrement=True)
    username = Column(String, unique=True, index=True)
    password = Column(String)
    is_active = Column(Boolean, default=True)
    is_admin = Column(Boolean, default=False)
```



# Сохранение пользователей

```
@router.post("/register")
async def register_user(form_data: Annotated[OAuth2PasswordRequestForm, Depends()], db: SessionDep) -> User:
    existing_user = await db.execute(select(DBUser).where(DBUser.username == form_data.username))
    existing_user = existing_user.scalar_one_or_none()
    if existing_user:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="User already exists")

    db_user = DBUser(username=form_data.username, password=hash_password(form_data.password))
    db.add(db_user)
    await db.commit()
    await db.refresh(db_user)

    return User(
        id=db_user.id,
        username=db_user.username,
        is_active=db_user.is_active,
        is_admin=db_user.is_admin
    )
```

# Refresh token

## Недостатки JWT токена

- Нельзя отозвать до истечения
- Короткий срок жизни

Когда access token истекает отправляется запрос на refresh с токеном для обновления

```
def create_refresh_token() -> str:  
    return hashlib.sha256(os.urandom(32)).hexdigest()
```

# Refresh token

```
@router.post("/token")
async def init_session(db: SessionDep, form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> AccessToken:
    result = await db.execute(select(DBUser).where(DBUser.username == form_data.username))
    user = result.scalar_one_or_none()
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid username or password"
        )

    if not validate_credentials(form_data.password, user.password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid username or password"
        )

    token = create_access_token(username=user.username, is_admin=user.is_admin, is_active=user.is_active, id=user.id)
    refresh_token = create_refresh_token()
    db_session = Session(user_id=user.id, token=refresh_token, expires_at=(datetime.now(timezone.utc) + timedelta(days=30)).replace(tzinfo=None))
    db.add(db_session)
    await db.commit()
    await db.refresh(db_session)
    return AccessToken(value=token, refresh_token=refresh_token)
```

# Refresh token

```
@router.post("/refresh")
async def refresh_token(db: SessionDep, request: RefreshTokenRequest) -> AccessToken:
    db_session = await db.execute(select(DBSession).where(DBSession.token == request.refresh_token))
    db_session = db_session.scalar_one_or_none()
    if not db_session:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid refresh token")

    user = await db.execute(select(DBUser).where(DBUser.id == db_session.user_id))
    user = user.scalar_one_or_none()
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="User not found")

    return AccessToken(value=create_access_token(username=user.username, is_admin=user.is_admin, is_active=user.is_active, id=user.id), refresh_token=create_refresh_token())
```

# Refresh token

```
@router.post("/logout")
async def logout(db: SessionDep, request: RefreshTokenRequest):
    db_session = await db.execute(select(DBSession).where(DBSession.token == request.refresh_token))
    db_session = db_session.scalar_one_or_none()
    if not db_session:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid refresh token")
    await db.delete(db_session)
    await db.commit()
    return {
        "detail": "Logged out"
    }
```

# Refresh token

## Итого

- Лучше хранить refresh token с информацией о user агенте
- Лучше хранить историю устаревших токенов, чтобы заводить инцидент
- Их нельзя хранить в localStorage
- Не надо логироваться каждые 5 минут

# Refresh token

## Итого

- Лучше хранить refresh token с информацией о user агенте
- Лучше хранить историю устаревших токенов, чтобы заводить инцидент
- Их нельзя хранить в localStorage
- Не надо логироваться каждые 5 минут

# SSO

это механизм единого входа, который позволяет пользователю один раз пройти аутентификацию и затем получать доступ к нескольким независимым приложениям или сервисам без повторного входа.





# Примеры SSO

- Google
- GitHub
- VK
- Keycloak (корпоративный)

# Google SSO

1. Создать новый проект в Google Cloud Console
2. Создать OAuth 2.0 Client ID с информацией о приложении и указать url для редиректа
3. Получить Client Id и Client Secret



# Google SSO FastAPI

```
google_sso = GoogleSSO(  
    client_id=GOOGLE_CLIENT_ID,  
    client_secret=GOOGLE_CLIENT_SECRET,  
    redirect_uri=GOOGLE_REDIRECT_URI,  
)  
  
@app.get("/auth-google")  
async def auth_login():  
    return await google_sso.get_login_redirect()  
  
@app.get("/auth/google/callback")  
async def auth_callback(request: Request):  
    async with google_sso:  
        sso_user = await google_sso.verify_and_process(request)  
  
    return JSONResponse(  
        {  
            "provider": sso_user.provider,  
            "id": sso_user.id,  
            "email": sso_user.email,  
            "name": sso_user.display_name,  
            "picture": getattr(sso_user, "picture", None),  
        }  
    )
```

# Google SSO

В приложении

1. Создать роут для редиректа на гугл авторизацию
2. Создать ручку для обработки callback'a, получить информацию о пользователе



# Keycloak

бесплатное решение с открытым исходным кодом для управления идентификацией и доступом

- Интеграции с различными технологиями
- Управление пользователями и ролями
- Административная консоль
- 2FA



# Преимущества SSO

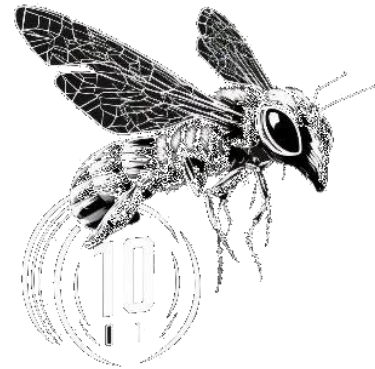
1. Сокращение забытых паролей
2. Один вход - множество ресурсов
3. Не нужно реализовывать логику логина, пароля, 2FA
4. Можно получить необходимые для себя данные пользователя

# OWASP TOP 10 API

<https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

## 1 - Broken Object Level Authorization

API позволяет пользователю получать доступ к данным, которые ему не принадлежат



# OWASP TOP 10 API

## 2 - Broken Authentication

Механизмы аутентификации реализованы неправильно, что позволяет злоумышленнику скомпрометировать учетные данные или токены аутентификации.

- Слабый токен
- Утечка ключей
- Отсутствие ограничений на попытки входа



# OWASP TOP 10 API

## 3 - Broken Object Property Level Authorization

Пользователь может изменять или читать отдельные поля объекта, к которым у него не должно быть доступа

# OWASP TOP 10 API

## 4 - Unrestricted Resource Consumption

Атаки, направленные на исчерпание ресурсов сервера (вычислительных, сетевых, дисковых), что приводит к замедлению работы или отказу в обслуживании (DoS).

# OWASP TOP 10 API

## 5 - Broken Function Level Authorization

Сложные иерархии ролей и прав реализованы неправильно, что позволяет обычным пользователям получать доступ к административным функциям.

Защита: запрещено по умолчанию

# OWASP TOP 10 API

## 6 - Unrestricted Access to Sensitive Business Flows

Атака на бизнес-логику. Злоумышленник может автоматизировать вызовы API, которые не предназначены для частого использования, чтобы получить бизнес-преимущество.

Защита: капчи, сложные лимиты запросов, аналитика поведения для выявления ботов

# OWASP TOP 10 API

## 7 - Server Side Request Forgery

API получает от пользователя URL и выполняет запрос к этому URL, не проводя должной валидации. Это позволяет злоумышленнику заставить сервер делать запросы к внутренним ресурсам (например, к метаданным облачного провайдера).

Защита: валидация

# OWASP TOP 10 API

## 8 - Security Misconfiguration

Ошибки в конфигурации компонентов стека технологий

- Невыключенный режим отладки
- Незакрытые ненужные методы
- Отсутствие заголовков безопасности

# OWASP TOP 10 API

## 9 - Improper Inventory Management

Отсутствие контроля за версиями и окружениями API. Старые, тестовые или заброшенные версии API остаются доступными и не защищены должным образом

# OWASP TOP 10 API

## 10 - Unsafe Consumption of APIs

При разработке собственного API вы используете сторонние API (например, для интеграции с платежной системой или погодным сервисом) без должной проверки их безопасности.



# Итого

1. Обсудили аутентификацию и авторизацию
2. Рассмотрели разные виды авторизации
3. Реализовали OAuth в FastAPI
4. Рассмотрели хранение паролей
5. Уменьши количество логинов посредством использования refresh токена
6. Рассмотрели SSO на примере Google
7. Рассмотрели топ уязвимостей АПИ

# Coding samples

<https://github.com/itmo-webdev/webdev-2025>



Вопросы?